

remote-operations 개요

`remote-operations` 는 관리 서버에서 FARM/LAB GPU 서버에 Wake-on-LAN 패킷을 보내고, 부팅된 서버의 SSH·공유 스토리지·GPU를 확인한 뒤 기존 사용자 컨테이너를 시작한다. 부팅 신호 전송부터 컨테이너 내부 SSH와 GPU 확인까지를 하나의 부팅 작업으로 묶어, 서버가 사용자 작업을 받을 수 있는 상태인지 확인하는 것이 목표다.

NIC·네트워크, Ansible 원격 접근, 서버의 공유 스토리지 mount와 GPU driver, 사용자 컨테이너는 미리 준비된 상태를 입력으로 사용한다. 이 모듈은 부팅 시 그 상태를 확인하고 제한된 복구를 수행하며, 해결되지 않은 실패를 로그와 알림으로 남긴다.

문서 구성

문서	핵심 내용
현재 페이지	remote-operations의 목표와 실행 전제
설계	실행 구조, 대상 서버 지정 방식, 부팅 신호 전송·서버 준비 상태 확인·컨테이너 기동 후 점검과 systemd 설계
운영	수동 실행, 모의 실행, 배포·점검과 장애 대응 절차
설정	대상 서버, 네트워크, 제한 시간, 공유 스토리지, 로그와 알림 설정 준비

remote-operations 설계

개요 · 운영 · 설정

GitHub 코드 링크: `admin_infra_server` 는 비공개 저장소다. 코드 링크를 누르면 GitHub 로그인 화면을 거쳐 원래 파일과 line으로 이동한다. 조직 저장소에 접근 권한이 있는 계정으로 로그인해야 한다.

1. 개요

`remote-operations` 는 관리 서버에서 FARM/LAB GPU 서버를 원격으로 깨운 뒤, 각 서버가 운영 가능한 상태인지 확인하고 기존 사용자 컨테이너를 시작하는 일련의 부팅 작업을 제공한다.

Wake-on-LAN 패킷을 전송한 것만으로는 서버를 사용할 수 있다고 판단할 수 없다. OS가 부팅되어 SSH가 열리고, 공유 스토리지와 NVIDIA driver가 준비된 뒤, 기존 컨테이너 안의 SSH와 GPU까지 동작해야 사용자 작업을 재개할 수 있다. 이 모듈은 이 과정을 하나의 제한 시간 안에서 순서대로 수행하고, 복구되지 않은 실패를 로그와 알림으로 남긴다.

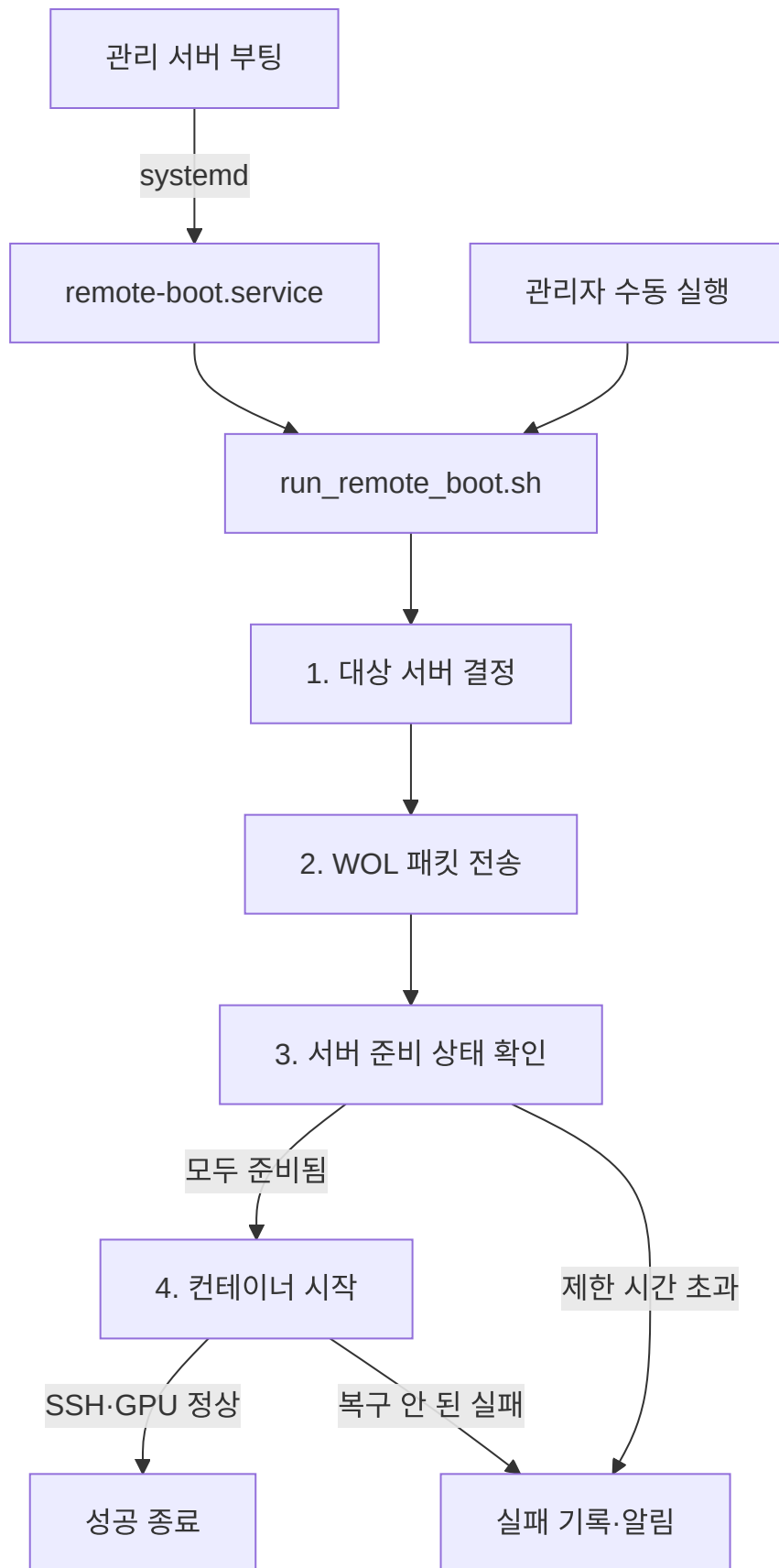
NIC·firmware의 Wake-on-LAN 설정, 네트워크 전달, Ansible inventory, 서버 mount와 GPU driver, 사용자 컨테이너 정의는 미리 준비되어 있어야 한다. `remote-operations` 는 이러한 설정들이 이미 되어 있다는 전제로, 부팅 대상으로 지정된 서버의 상태를 확인하고, 안전하게 제한할 수 있는 복구만 수행한다.

2. 실행 구조

`remote-operations` 의 모든 로직은 하나의 스크립트 `run_remote_boot.sh` 에 들어 있다. 이 스크립트를 실행하는 경로는 두 가지다.

- **자동:** 관리 서버가 부팅되면 systemd가 `remote-boot.service` 를 실행한다. 이 service가 하는 일은 `run_remote_boot.sh` 를 한 번 호출하는 것뿐이다 (service를 만드는 방법은 6절 참고).
- **수동:** 관리자가 같은 스크립트를 직접 실행한다.

`run_remote_boot.sh` 는 실행되면 아래 다섯 단계를 순서대로 진행한다.



각 단계를 담당하는 스크립트는 다음과 같다.

단계	담당 스크립트	수행하는 작업
대상 서버 결정	<code>run_remote_boot.sh</code> , <code>common.sh</code>	설정 또는 명령행 입력을 구체적인 서버 ID 목록으로 변환한다.
WOL 패킷 전송	<code>wake_targets.sh</code>	각 서버의 MAC과 broadcast IP로 Wake-on-LAN 패킷을 보낸다.
서버 준비 상태 확인	<code>wait_for_priority_servers.sh</code> , <code>check_server_boot_health.sh</code>	선택한 모든 서버에서 SSH·공유 스토리지·GPU가 준비될 때까지 다시 확인한다.
컨테이너 시작	<code>restart_all_remote_containers.sh</code>	대상 이미지의 기존 컨테이너를 시작하고 내부 SSH·GPU를 확인한다.
실패 기록·알림	<code>common.sh</code>	단계별 로그를 남기고 같은 내용의 반복 전송을 억제한 알림을 보낸다.

`common.sh`는 특정 단계 전용이 아니라 여러 단계가 공통으로 불러 쓰는 유틸리티 스크립트다. 대상 서버 변환(3절), Ansible 원격 실행과 로그·알림 (4.5절)에서 반복해서 쓰인다.

위 표가 각 단계의 담당 스크립트를 보여준다면, 아래는 그 단계들이 실행되는 시간 흐름과 조건이다. 실행 순서는 다음과 같다.

1. 설정된 사전 대기 시간이 지나면 대상 서버 전체에 WOL 패킷을 전송한다.
2. 서버 준비 상태 확인이 활성화되어 있으면 선택한 모든 서버의 SSH·공유 스토리지·GPU가 준비될 때까지 기다린다.
3. 하나라도 제한 시간 안에 준비되지 않으면 컨테이너 시작으로 넘어가지 않고 실패를 기록한다.
4. 모든 서버가 준비되면 선택한 서버에서 중지되어 있던 대상 컨테이너를 시작한다.
5. 각 대상 컨테이너의 SSH와 GPU를 확인하고, 복구되지 않은 실패가 있으면 전체 실행을 실패로 종료한다.

서버 준비 상태 확인과 컨테이너 시작 단계는 설정으로 각각 끌 수 있다. 기본 실행에서는 두 단계를 모두 사용하여 서버가 준비되기 전에 사용자 컨테이너가 시작되는 것을 막는다.

관련 코드

- `run_remote_boot.sh`: 설정 읽기, 대상 서버 확장, WOL, 서버 준비 상태 확인과 컨테이너 시작 단계를 조합한다.
- `install_remote_boot_service.sh`: 관리 서버 부팅 시 전체 부팅 작업을 호출하는 systemd unit을 만든다.

3. 대상 서버 지정과 원격 접근

실행 대상은 다음과 같이 지정한다.

입력	선택되는 서버
<code>FARM1</code> , <code>LAB10</code>	지정한 서버 한 대
<code>all-farm</code> , <code>all-lab</code>	설정에 등록된 FARM 또는 LAB 서버 전체
<code>all</code>	설정에 등록된 FARM과 LAB 서버 전체

입력	선택되는 서버
명령행 입력 생략	REMOTE_BOOT_TARGETS에 설정한 기본 대상

`common.sh`는 입력된 서버 ID와 그룹 이름을 실제 서버 ID 목록으로 변환하고 중복을 제거한다. 이후의 부팅 신호 전송과 상태 확인은 이 목록을 기준으로 실행한다.

서버 ID는 두 종류의 연결 정보에 사용된다.

목적	사용하는 정보
WOL 패킷 전송	서버별 MAC address와 FARM/LAB broadcast IP
상태 확인과 원격 명령	서버 ID에서 변환한 Ansible inventory 별칭

부팅 신호를 보내는 네트워크 주소와 OS 부팅 후 접속하는 Ansible 주소를 분리했기 때문에 SSH 주소나 port가 바뀌어도 `FARM1`과 같은 운영 ID는 유지할 수 있다. 반대로 두 정보가 같은 물리 서버를 가리키도록 설정되어 있어야 한다.

최종 서버 ID 목록을 모든 단계가 공통으로 사용하므로 단일 서버, 한 구역 전체와 전체 서버 실행이 같은 절차를 따른다.

관련 코드

- `common.sh`의 대상 서버 목록 변환: 대상 문자열을 정리하고 그룹 이름을 구체적인 서버 목록으로 변환한다.
- `wake_targets.sh`의 주소 선택: 서버 ID에 해당하는 MAC과 구역별 broadcast IP를 선택한다.
- `common.sh`의 Ansible 실행: 서버 ID를 inventory 별칭으로 변환하고 원격 shell을 실행한다.

4. 부팅 단계별 설계

4.1 Wake-on-LAN

`wake_targets.sh`는 선택한 각 서버의 MAC address를 확인하고 FARM/LAB 네트워크에 맞는 broadcast IP로 magic packet을 전송한다. 서버별 상태 확인을 중간에 수행하지 않고 모든 패킷을 먼저 보낸 뒤 서버 준비 상태 확인 단계로 이동한다.

WOL 명령의 성공은 패킷을 보냈다는 의미다. 실제 전원 인가, firmware 부팅과 OS 준비는 확인하지 못하므로 다음 단계에서 먼저 SSH 접속 가능 여부를 확인한다. 서버의 MAC이 없거나 알 수 없는 ID가 들어오면 패킷을 보내지 않고 실패한다.

관련 코드

- `send_magic_packet`: 서버별 WOL 명령을 실행하거나 모의 실행 계획을 기록한다.

4.2 전체 서버 준비 상태 확인

이 단계는 선택한 모든 서버가 준비된 뒤에만 컨테이너 시작을 허용한다. 아직 준비되지 않은 서버만 목록에 남겨 설정된 주기마다 다시 확인하며, 한 번 준비가 확인된 서버는 같은 실행에서 다시 검사하지 않는다.

제한 시간은 서버마다 따로 적용되지 않고 선택한 서버 전체에 한 번 적용된다. 한 서버라도 제한 시간까지 준비되지 않으면 확인 단계가 실패하고 컨테이너 시작 단계는 실행되지 않는다. 이를 통해 일부 서버만 준비된 상태에서 사용자 컨테이너가 먼저 올라오는 것을 막는다.

`wait_for_priority_servers.sh` 라는 파일명은 과거 명칭이 남아 있는 것이며, 현재 구현은 우선순위 서버를 따로 선택하지 않고 전달된 모든 대상 서버를 동일하게 확인한다.

관련 코드

- `wait_for_priority_servers.sh`: 준비가 확인된 서버와 아직 확인 중인 서버를 구분하고 전체 제한 시간을 관리한다.

4.3 서버 상태 확인과 제한된 복구

`check_server_boot_health.sh` 는 각 서버의 상태를 다음 순서대로 확인한다.

확인 항목	확인 방법	실패했을 때 수행하는 복구
SSH	Ansible remote shell로 <code>true</code> 실행	설정된 횟수만큼 연결을 다시 시도한다.
공유 스토리지	지정한 source가 서버별 mount 경로에 연결됐는지 <code>findmnt</code> 로 확인	해당 경로의 mount 또는 <code>mount -a</code> 를 한 번 시도한 뒤 다시 확인한다.
서버 GPU	서버에서 <code>nvidia-smi</code> 실행	NVIDIA module load와 persistence service 재시작을 한 번 시도한 뒤 다시 확인한다.

mount와 GPU 복구는 한 번의 상태 확인 과정에서 각각 한 번만 수행한다. 그래도 실패하면 서버가 아직 준비되지 않은 것으로 처리하며, 전체 제한 시간이 남아 있으면 해당 서버의 상태를 다시 확인한다.

이 복구는 이미 준비된 mount 설정과 NVIDIA driver를 다시 활성화하는 범위다. fstab, NFS 인증이나 driver package를 새로 구성하지 않는다. 따라서 설정 자체가 잘못된 경우에는 재시도로 숨기지 않고 실패로 남겨 해당 운영 절차에서 수정할 수 있게 한다.

관련 코드

- `run_step_with_single_recovery`: mount와 GPU를 확인하고 한 번의 복구 후 재확인한다.
- `check_server_boot_health.sh` 의 실행 순서: 서버 ID를 mount·Ansible 정보로 변환하고 SSH, mount, GPU를 순서대로 확인한다.

4.4 기존 컨테이너 시작과 기동 후 점검

모든 서버의 준비가 확인되면 `restart_all_remote_containers.sh` 가 Docker 목록에서 설정한 이미지 정규식에 맞는 컨테이너만 선택한다. 기본값은 `decs` 또는 `dguailab/decs` 이미지를 대상으로 한다.

선택된 컨테이너 중 중지된 항목을 시작하고, 이미 실행 중인 항목도 포함해 다음 기동 후 점검을 수행한다.

1. 컨테이너가 실행 상태인지 확인한다.
2. 컨테이너 내부 SSH service를 확인하고, 준비되지 않았으면 service 시작을 시도한다.
3. 컨테이너 내부에서 `nvidia-smi` 를 실행한다.
4. GPU 확인이 처음 실패하면 컨테이너를 한 번 재시작하고 제한 시간 동안 다시 확인한다.

중지된 컨테이너의 일괄 시작이 실패하면 Docker service를 한 번 재시작한 뒤 다시 시도한다. 서버별 처리가 실패하면 전체 제한 시간이 남아 있는 동안 해당 서버를 다시 처리한다.

사용자 계정, mount, port와 Docker option은 기존 컨테이너에 이미 들어 있다. 이 모듈은 그 정보를 다시 결정하거나 컨테이너를 재생성하지 않고, 기존 컨테이너를 시작하고 준비 상태만 확인한다. 임시 GPU 컨테이너를 만드는 `create_test_container.sh`와 `delete_test_container.sh`는 독립 점검 도구이며 이 부팅 흐름에서는 호출하지 않는다.

관련 코드

- `restart_all_remote_containers.sh`의 대상 선택: 이미지 정규식과 Docker 목록으로 처리할 컨테이너를 고른다.
- `restart_remote_containers`: 중지된 컨테이너 시작과 SSH·GPU 기동 후 점검을 수행한다.
- 서버별 재시도와 제한 시간: 실패한 서버만 남겨 전체 제한 시간 안에서 재시도한다.

4.5 로그와 실패 알림

각 스크립트는 시각, 실행 구분값, 단계, 서버와 실패 원인을 포함한 로그를 남긴다. 서버 상태 확인과 컨테이너 기동 후 점검 로그는 실행별 파일로도 저장할 수 있어 systemd 전체 로그와 개별 실패 근거를 나누어 확인할 수 있다.

실패 알림은 내부 알림 API를 통해 Slack으로 전달한다. Slack이 설정되지 않았거나 전송이 실패하면 대체 로그에 남긴다. 같은 실패 내용을 기준으로 만든 상태 파일이 이미 있으면 반복 알림을 억제한다. 서버 상태 확인과 컨테이너 단계가 다시 성공하면 각각 관련 상태 파일을 지운다. 실행 로그와 알림 억제 상태를 분리하여 과거 로그를 지우지 않고도 중복 알림 상태만 초기화할 수 있다.

관련 코드

- `common.sh`의 알림 상태 관리: 실패 내용별 상태 파일을 만들고 성공한 단계의 상태를 정리한다.
- `send_slack_message`와 `notify_failure`: 내부 알림 API 전송, 대체 로그 기록과 중복 억제를 처리한다.

5. 설정 모델

실제 실행값은 Git에서 제외된 `config/remote_boot.local.env`에 두고, 저장소에는 변수와 기본 구조만 보여 주는 `config/remote_boot.example.env`를 유지한다. 설정은 다음 영역으로 나뉜다.

설정 영역	결정하는 내용
대상 서버	FARM/LAB 서버 목록과 기본 실행 대상
WOL 네트워크	서버별 MAC과 구역별 broadcast IP
서버 준비 상태 확인	확인 단계 사용 여부, 전체 제한 시간과 재확인 주기
서버 상태 기준	요구하는 NFS source와 서버별 mount 경로
컨테이너	시작 단계 사용 여부, 대상 이미지 정규식과 기동 후 점검 제한 시간
로그·알림	상태 확인 로그 위치, service 로그 보존, 알림 상태와 Slack 전달

스크립트는 설정값을 읽되 MAC, webhook 같은 실제 값은 예제나 source에 넣지 않는다. 설정 파일의 각 항목과 준비 방법은 [설정 문서](#)에서 설명한다.

관련 코드

- `remote_boot.example.env`: 실행 설정의 공개 가능한 구조와 기본값을 정의한다.

6. systemd 연동

설치 스크립트는 관리 서버에 `remote-boot.service` (systemd가 관리하는 서비스 정의 단위, 즉 unit)와 logrotate 설정을 만든다. logrotate는 systemd와 별개인 리눅스 표준 도구로, 로그 파일이 계속 커지지 않도록 주기적으로 나누고 오래된 로그를 정리한다. service는 `network-online.target` 이후 설치를 실행한 사용자 권한으로 `run_remote_boot.sh`를 호출하고, 출력은 service 로그에 추가한다.

이 unit의 `Type`은 `oneshot`이다. `Type`은 systemd가 서비스의 실행 방식을 구분하는 값이고, `oneshot`은 계속 떠 있는 daemon이 아니라 작업을 한 번 수행하고 종료되는 타입이다. 부팅 작업이 완료되면 process가 종료되며 상태를 계속 관찰하지 않는다. 따라서 제한 시간과 재시도는 이번 부팅 작업을 완료하기 위한 범위로 제한되고, 이후의 지속적인 서버 상태 관측과 알림은 `monitoring`에서 수행한다.

설치 스크립트를 다시 실행하면 생성할 unit과 현재 파일을 비교해 필요한 경우에만 갱신하고, `systemctl daemon-reload`와 `enable`를 적용한다. service 로그에는 daily logrotate와 보존 개수가 함께 설정된다.

관련 코드

- `install_remote_boot_service.sh`: oneshot unit, logrotate와 enable 동작을 구성한다.

7. 디렉터리 지도

경로	역할
<code>script/run_remote_boot.sh</code>	전체 부팅 작업의 시작점
<code>script/wake_targets.sh</code>	대상 서버별 Wake-on-LAN 패킷 전송
<code>script/wait_for_priority_servers.sh</code>	선택한 모든 서버의 준비 상태 확인
<code>script/check_server_boot_health.sh</code>	SSH·mount·서버 GPU 확인과 제한된 복구
<code>script/restart_all_remote_containers.sh</code>	기존 대상 컨테이너 시작과 SSH·GPU 기동 후 점검
<code>script/common.sh</code>	대상 서버, Ansible, 로그와 알림 공통 함수
<code>script/install_remote_boot_service.sh</code>	systemd unit과 logrotate 설치
<code>script/reset_remote_boot_alert_state.sh</code>	중복 알림 억제 상태 초기화
<code>script/create_test_container.sh</code> , <code>script/delete_test_container.sh</code>	부팅 흐름과 분리된 임시 GPU 컨테이너 점검 도구
<code>config/remote_boot.example.env</code>	공개 가능한 설정 구조와 기본값
<code>config/remote_boot.local.env</code>	실제 환경값을 두는 Git 제외 설정 파일
<code>test/dry_run_remote_boot.sh</code>	WOL, 서버 상태, 컨테이너와 전체 흐름 모의 실행

경로	역할
test/integration_smoke_test.sh	Ansible·Docker·GPU 통합 확인
test/test_slack_notification.sh	내부 알림 API와 Slack 알림 경로 확인

remote-operations 운영

개요 · 설계 · 설정

1. 개요

이 문서의 목표는 `remote-operations`를 관리 서버에 배포하고, 변경 내용을 실제 서버에 적용하기 전에 검증하며, WOL·서버 상태 확인·컨테이너 시작 단계를 목적에 맞게 실행하고 실패 원인을 확인하는 절차를 제공하는 것이다.

운영은 다음 순서를 기준으로 한다.

- 로컬 설정과 Ansible 원격 접근을 준비한다.
- 대상 서버와 실행 계획을 모의 실행으로 확인한다.
- WOL, 서버 상태 확인과 컨테이너 단계를 개별 서버에서 검증한다.
- 전체 흐름을 수동 실행해 로그와 알림을 확인한다.
- systemd unit을 설치하거나 갱신해 관리 서버 부팅에 연결한다.

모든 명령은 다음 directory에서 실행한다.

```
cd /home/jy/server_manage/remote-operations
```

2. 목적별 실행 위치

운영 목적	사용할 명령 또는 파일
사용 가능한 대상 서버를 확인한다.	<code>./script/wake_targets.sh --list-targets</code>
WOL 패킷만 보낸다.	<code>./script/wake_targets.sh TARGET...</code>
한 서버의 SSH·mount·GPU를 확인한다.	<code>./script/check_server_boot_health.sh --server-id SERVER_ID</code>
기존 대상 컨테이너를 시작하고 SSH·GPU를 확인한다.	<code>./script/restart_all_remote_containers.sh SERVER_ID...</code>
WOL부터 컨테이너 기동 후 점검까지 전체 흐름을 실행한다.	<code>./script/run_remote_boot.sh TARGET...</code>
관리 서버 부팅에 전체 흐름을 연결한다.	<code>./script/install_remote_boot_service.sh</code>

운영 목적	사용할 명령 또는 파일
Slack 알림 전달을 확인한다.	<code>./test/test_slack_notification.sh</code>
저장된 중복 알림 억제 상태를 초기화한다.	<code>./script/reset_remote_boot_alert_state.sh</code>

3. 실행 전 준비

실제 서버에 접근하기 전에 다음 조건을 확인한다.

- `config/remote_boot.local.env` 가 준비되어 있다.
- 대상 서버의 MAC과 broadcast IP가 실제 네트워크와 일치한다.
- 각 서버의 Wake-on-LAN이 firmware와 NIC에서 활성화되어 있다.
- 설정한 Ansible inventory로 대상 서버에 SSH 접속할 수 있다.
- mount와 GPU 복구에 필요한 원격 명령을 `sudo -n` 으로 실행할 수 있다.
- 서버의 NFS mount와 NVIDIA driver가 이미 구성되어 있다.
- 컨테이너 시작 대상 이미지 정규식이 실제 사용자 컨테이너와 일치한다.

설정 파일을 처음 만드는 절차와 변수 의미는 [설정 문서](#)를 따른다.

4. 변경 전 검증

4.1 대상 서버 확인

```
./script/wake_targets.sh --list-targets
./script/run_remote_boot.sh --list-targets
```

로컬 설정의 FARM/LAB 목록과 `all-farm`, `all-lab`, `all` 확장 결과를 확인한다. 새 서버를 추가했다면 이 목록에 먼저 나타나야 한다.

4.2 모의 실행

`dry_run_remote_boot.sh`의 첫 번째 인자는 어느 단계를 모의 실행할지 고르는 모드다. 각각 [설계 문서](#) 2절의 단계에 대응한다: `wake`는 WOL 패킷 전송, `health`는 서버 준비 상태 확인, `containers`는 컨테이너 시작, `full`은 전체 부팅 흐름이다.

```
./test/dry_run_remote_boot.sh wake FARM1 LAB1
./test/dry_run_remote_boot.sh health FARM1
./test/dry_run_remote_boot.sh containers FARM1
./test/dry_run_remote_boot.sh full FARM1 LAB1
```

`wake`, `health`, `full` 모의 실행은 WOL 패킷, 대기과 상태 변경 명령을 실행하지 않고 계획을 출력한다. `containers` 모의 실행은 실제 컨테이너를 시작하거나 재시작하지 않지만 현재 Docker 목록을 읽어 처리 대상과 기동 후 점검 계획을 만든다. 따라서 컨테이너 모의 실행에도 Ansible inventory와 조회 목적의 원격 연결이 필요하다.

출력에서 다음을 확인한다.

- 입력한 그룹 이름이 의도한 구체적 서버 ID로 확장되는가?
- 각 ID의 MAC, broadcast IP와 Ansible 서버 별칭이 같은 장비를 가리키는가?
- 구역별 요구 mount source와 서버별 mount 경로가 올바른가?
- 대상 이미지 정규식이 시작해야 할 컨테이너만 선택하는가?
- 제한 시간과 재확인 주기가 선택한 서버 수와 실제 부팅 시간을 감당하는가?

4.3 실제 연결 확인

```
./test/integration_smoke_test.sh
```

이 명령은 선택된 서버에 Ansible ping, Docker daemon과 서버 GPU 확인을 실제로 수행한 뒤 서버 준비 상태를 확인한다. 이 과정은 mount와 GPU가 실패하면 제한된 복구도 수행하므로 단순 조회 명령이 아니다.

`--full-flow`를 추가하면 WOL부터 컨테이너 시작까지 실제 전체 흐름을 실행한다. 변경 검증의 첫 단계에서는 사용하지 않고, 개별 단계가 통과한 뒤 명시적으로 실행한다.

```
./test/integration_smoke_test.sh --full-flow
```

4.4 Slack 알림 확인

```
./test/test_slack_notification.sh --server-id FARM1
```

이 명령은 내부 알림 API를 통해 실제 Slack message를 전송한다. 확인 전에 `REMOTE_BOOT_SLACK_ENABLED`와 webhook 설정을 확인하고 운영 채널에 시험 메시지가 전송된다는 점을 공유한다.

5. 수동 실행

5.1 WOL만 실행

```
./script/wake_targets.sh FARM1
./script/wake_targets.sh all-farm
./script/wake_targets.sh FARM1 LAB1
```

명령 성공은 magic packet을 전송했다는 의미이며 서버 부팅 완료를 보장하지 않는다. 이후 준비 상태 확인으로 실제 SSH·mount·GPU를 확인한다.

5.2 한 서버의 준비 상태 확인

```
./script/check_server_boot_health.sh --server-id FARM1
```

SSH, 요구 mount와 서버의 `nvidia-smi` 를 순서대로 확인한다. mount 실패 시 `mount` 또는 `mount -a`, GPU 실패 시 module load와 persistence service 재시작을 시도하므로 실제 서버 상태가 바뀔 수 있다.

5.3 기존 컨테이너 시작과 기동 후 점검

```
./script/restart_all_remote_containers.sh FARM1
```

설정된 이미지 정규식에 맞는 기존 컨테이너만 처리한다. 중지된 컨테이너를 시작한 뒤 컨테이너 내부 SSH와 GPU를 확인하며, 실패 시 Docker service 또는 컨테이너 재시작이 발생할 수 있다.

5.4 전체 부팅 작업

```
./script/run_remote_boot.sh FARM1 LAB1  
./script/run_remote_boot.sh --targets "all-farm"
```

명령행에서 지정한 대상은 `REMOTE_BOOT_TARGETS` 기본값을 대체한다. 실제 전체 서버를 대상으로 실행하기 전에 한 서버, 한 구역 순서로 범위를 늘린다.

6. systemd 배포와 갱신

6.1 최초 설치

```
./script/install_remote_boot_service.sh
```

설치 스크립트는 다음 항목을 적용한다.

- `/etc/systemd/system/remote-boot.service`
- `/etc/logrotate.d/remote-boot`
- service 로그 파일과 소유권
- `systemctl daemon-reload` 와 service enable

설치 스크립트를 실행한 사용자와 group이 oneshot service를 실행한다. 해당 계정이 repository, 로컬 설정, Ansible inventory와 SSH key를 읽을 수 있어야 한다.

설치 직후 실행까지 확인하려면 `--start-now` 를 사용한다.

```
./script/install_remote_boot_service.sh --start-now
```

6.2 변경 후 재설치가 필요한 경우

`run_remote_boot.sh`를 비롯한 source 스크립트와 기존 로컬 설정값은 service가 다음 실행 때 같은 경로에서 다시 읽으므로 일반적으로 unit 재설치가 필요하지 않다. 다음 항목을 바꾸면 설치 스크립트를 다시 실행한다.

- 저장소 또는 실행 스크립트 경로
- service 실행 사용자와 group
- 설정 파일 경로
- service 로그 경로나 logrotate 보존 개수
- systemd unit의 dependency나 실행 option

unit 내용을 강제로 다시 쓰려면 `--force`를 사용한다.

```
./script/install_remote_boot_service.sh --force
```

6.3 배포 확인

```
systemctl is-enabled remote-boot.service  
systemctl status remote-boot.service  
journalctl -u remote-boot.service -b  
tail -f /var/log/remote-boot.log
```

`Type=oneshot`이므로 실행을 완료한 뒤 계속 실행 중인 daemon process는 없다. exit status와 이번 boot의 log로 성공 여부를 판단한다.

7. 새 서버 추가

1. 서버의 Wake-on-LAN을 firmware와 NIC에서 활성화하고 실제 MAC을 확인한다.
2. 관리 서버의 Ansible inventory에 해당 서버 별칭을 추가하고 SSH 접속을 검증한다.
3. `REMOTE_BOOT_FARM_TARGETS` 또는 `REMOTE_BOOT_LAB_TARGETS`에 서버 ID를 추가한다.
4. `REMOTE_BOOT_MAC_<SERVER_ID>`에 MAC을 설정한다.
5. 구역별 broadcast IP와 요구 mount source, 서버별 mount 경로를 확인한다.
6. `--list-targets`와 `wake` 모의 실행으로 대상 해석과 WOL 명령을 확인한다.
7. 실제 WOL 후 해당 서버 하나의 준비 상태를 확인한다.
8. 기존 대상 컨테이너가 있는 경우 컨테이너 모의 실행과 기동 후 점검을 수행한다.
9. 마지막으로 전체 부팅 작업을 단일 서버 대상으로 모의 실행하고 실제 실행한다.

새 서버를 대상 목록에 추가하는 것만으로 Ansible 접근, mount, GPU driver나 컨테이너가 구성되지는 않는다. 각 영역의 준비가 완료된 뒤 이 모듈의 검증 절차를 연결한다.

8. 실패 진단

실패 단계 또는 원인	먼저 확인할 내용
wake_failed	MAC 누락, wakeonlan 설치, broadcast IP와 관리 네트워크
ssh_connection_failed	실제 전원 상태, inventory host, SSH key·사용자와 boot 시간
mount_unavailable	findmnt source/target, fstab, NFS·Kerberos 상태와 sudo -n
host_gpu_unavailable	서버의 nvidia-smi, module, persistence service와 driver 로그
docker_start_failed	Docker daemon, 대상 컨테이너 상태와 서버 resource
ssh_unavailable	컨테이너 로그, entrypoint와 컨테이너 내부 sshd
gpu_unavailable	컨테이너 image, NVIDIA runtime, 할당 GPU와 컨테이너 로그
Slack 전송 실패	내부 알림 API 도달성, webhook 설정과 대체 로그

서버 준비 상태 확인이 실패하면 제한 시간을 늘리기 전에 어느 단계에서 대기 중인지 확인한다. mount·GPU 설정 자체가 잘못되었거나 컨테이너 정의가 오래된 경우에는 해당 설정을 수정한 뒤 단일 서버부터 다시 검증한다.

9. 중복 알림 억제 상태 초기화

같은 실패가 이미 전송되어 새 알림이 억제되는 경우, 저장된 메시지를 확인한 뒤 필요한 범위만 초기화한다.

```
./script/reset_remote_boot_alert_state.sh --server-id FARM1
./script/reset_remote_boot_alert_state.sh --stage mount_check
./script/reset_remote_boot_alert_state.sh --reason mount_unavailable
```

여러 조건을 같이 주면 모두 일치하는 상태 파일만 지운다. 전체 상태 파일 삭제는 현재 실패 상태와 알림 재전송 영향을 확인한 뒤 실행한다.

```
./script/reset_remote_boot_alert_state.sh --all
```

이 명령은 실행 로그를 지우지 않고 중복 알림 억제를 상태 파일만 삭제한다.

10. 운영 안전 수칙

- 실제 실행 전에 같은 대상 서버로 모의 실행을 확인한다.

- `all` 실행보다 단일 서버와 구역 단위 검증을 먼저 수행한다.
- 로컬 설정의 MAC, SSH 경로와 webhook을 source나 로그에 복사하지 않는다.
- mount와 GPU 복구가 상태를 변경한다는 점을 고려해 maintenance 범위를 정한다.
- 실제 사용자 컨테이너를 임시 시험용 컨테이너로 대체하거나 재생성하지 않는다.
- 지속적으로 반복되는 장애는 부팅 재시도로 숨기지 않고 해당 모듈의 운영 절차에서 원인을 수정한다.

remote-operations 설정

개요 · 설계 · 운영

1. 개요

이 문서는 `remote-operations` 가 부팅 신호 전송, 서버 준비 상태 확인, 컨테이너 기동 후 점검과 알림에 사용하는 설정을 준비하는 방법을 설명한다. 실제 환경값은 Git에서 제외된 `config/remote_boot.local.env` 에 두고, 저장소의 `config/remote_boot.example.env` 에는 변수 구조와 공개 가능한 기본값만 유지한다.

로컬 설정 파일을 만든다.

```
cd /home/jy/server_manage/remote-operations
cp config/remote_boot.example.env config/remote_boot.local.env
chmod 600 config/remote_boot.local.env
```

실제 MAC, 개인 경로와 Slack webhook을 example 파일이나 Git commit에 넣지 않는다.

2. 대상 서버 설정

```
REMOTE_BOOT_FARM_TARGETS="FARM1 FARM2"
REMOTE_BOOT_LAB_TARGETS="LAB1 LAB2 LAB10"
REMOTE_BOOT_TARGETS="all"
```

변수	의미
<code>REMOTE_BOOT_FARM_TARGETS</code>	<code>all-farm</code> 이 나타내는 FARM 서버 ID 목록
<code>REMOTE_BOOT_LAB_TARGETS</code>	<code>all-lab</code> 이 나타내는 LAB 서버 ID 목록
<code>REMOTE_BOOT_TARGETS</code>	명령행에서 대상을 지정하지 않았을 때 사용할 기본 대상

서버 ID는 `FARM<number>` 또는 `LAB<number>` 형식을 사용한다. 목록에 추가된 ID는 WOL MAC과 Ansible inventory의 서버가 같은 물리 서버를 가리켜야 한다.

3. WOL 네트워크 설정

```
REMOTE_BOOT_FARM_BROADCAST_IP="192.168.2.255"
REMOTE_BOOT_LAB_BROADCAST_IP="192.168.1.255"

REMOTE_BOOT_MAC_FARM1="<FARM1 MAC>"
REMOTE_BOOT_MAC_LAB1="<LAB1 MAC>"
```

구역별 broadcast IP는 관리 서버에서 해당 네트워크로 WOL 패킷을 보낼 수 있는 주소여야 한다.

`REMOTE_BOOT_MAC_<SERVER_ID>` 에는 서버의 WOL 대상 NIC MAC을 넣는다. MAC은 장비 관리 기준 문서나 실제 서버에서 확인하며 임의로 추정하지 않는다.

설정 파일만으로 Wake-on-LAN이 활성화되지는 않는다. 서버 firmware, NIC와 네트워크의 broadcast 전달 설정은 별도로 준비되어 있어야 한다.

4. Ansible 원격 접근

`remote-operations` 는 서버 ID를 소문자 Ansible 별칭으로 바꾸어 사용한다. 예를 들어 `FARM1` 은 inventory의 `farm1` , `LAB10` 은 `lab10` 에 연결된다.

기본 Ansible 설정을 사용하지 않을 경우 로컬 설정에 개인 inventory를 지정한다.

```
REMOTE_BOOT_ANSIBLE_INVENTORY="/home/<관리자계정>/ansible/inventory.ini"
```

inventory 예시는 다음과 같다.

```
[farm]
farm1 ansible_host=<FARM1 address> ansible_user=<관리자계정>

[lab]
lab10 ansible_host=<LAB10 address> ansible_user=<관리자계정>
```

inventory는 접속 주소와 사용자를 정할 뿐 SSH 권한을 부여하지 않는다. 관리 서버의 해당 service 사용자 SSH key가 원격 계정의 `authorized_keys` 에 등록되어 있어야 한다. mount와 GPU 복구에는 대상 서버에서 password 입력 없이 실행할 수 있는 `sudo -n` 권한도 필요하다.

다음 명령으로 각 alias를 먼저 확인한다.


```
ansible farm1 -i /home/<관리자계정>/ansible/inventory.ini -m ping
ansible lab10 -i /home/<관리자계정>/ansible/inventory.ini -m ping
```

5. 서버 준비 상태 확인과 제한 시간

```
REMOTE_BOOT_PRE_DELAY_SECONDS=0
REMOTE_BOOT_ENABLE_GATE=true
REMOTE_BOOT_GATE_TIMEOUT_SECONDS=360
REMOTE_BOOT_GATE_POLL_SECONDS=20
```

변수	의미
REMOTE_BOOT_PRE_DELAY_SECONDS	관리 서버의 네트워크 준비 후 WOL을 보내기 전 대기 시간
REMOTE_BOOT_ENABLE_GATE	컨테이너 시작 전에 모든 서버의 준비 상태를 확인할지 결정
REMOTE_BOOT_GATE_TIMEOUT_SECONDS	선택된 서버 전체의 준비 상태를 확인할 제한 시간
REMOTE_BOOT_GATE_POLL_SECONDS	아직 준비되지 않은 서버를 다시 확인할 간격

제한 시간은 서버마다 새로 시작되지 않는다. 동시에 선택한 서버 수와 가장 느린 실제 부팅 시간을 기준으로 정한다. 원인을 확인하지 않은 채 제한 시간만 계속 늘리지 않는다.

6. 서버 상태 확인 기준

```
REMOTE_BOOT_FARM_REQUIRED_MOUNT="<FARM NFS source>"
REMOTE_BOOT_LAB_REQUIRED_MOUNT="<LAB NFS source>"
REMOTE_BOOT_HOST_SHARE_MOUNT_TEMPLATE="/home/tako%s/share"
```

REMOTE_BOOT_*_REQUIRED_MOUNT 는 findmnt 에서 일치해야 하는 source다. REMOTE_BOOT_HOST_SHARE_MOUNT_TEMPLATE 의 %s 는 서버 번호로 바뀐다. 예를 들어 FARM6 은 /home/tako6/share 를 검사한다.

이 값은 기존 서버 mount 설정을 확인하기 위한 기준이다. NFS export, fstab이나 Kerberos 인증을 생성하지 않는다. 실제 findmnt -rn -T <path> -o SOURCE,TARGET 출력과 동일한 source를 사용한다.

7. 컨테이너 시작과 기동 후 점검

```
REMOTE_BOOT_ENABLE_CONTAINER_RESTART=true
REMOTE_BOOT_CONTAINER_RESTART_TIMEOUT_SECONDS=600
REMOTE_BOOT_CONTAINER_RESTART_POLL_SECONDS=20
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_TIMEOUT_SECONDS=60
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_POLL_SECONDS=5
```

변수	의미
REMOTE_BOOT_ENABLE_CONTAINER_RESTART	서버 준비 상태 확인 후 기존 대상 컨테이너를 시작할지 결정
REMOTE_BOOT_CONTAINER_RESTART_TIMEOUT_SECONDS	선택한 서버 전체의 컨테이너 처리 제한 시간
REMOTE_BOOT_CONTAINER_RESTART_POLL_SECONDS	실패한 서버를 다시 처리할 간격
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_TIMEOUT_SECONDS	컨테이너별 SSH와 GPU 확인 제한 시간
REMOTE_BOOT_CONTAINER_POST_RESTART_CHECK_POLL_SECONDS	컨테이너 기동 후 점검을 다시 시도할 간격

처리 대상 이미지는 REMOTE_BOOT_CONTAINER_TARGET_IMAGE_REGEX 로 제한할 수 있다. 설정하지 않으면 decs 와 dguai Lab/decs repository를 대상으로 한다.

```
REMOTE_BOOT_CONTAINER_TARGET_IMAGE_REGEX='^(decs|dguai Lab/decs)(:|$)'
```

정규식을 바꿀 때는 컨테이너 모의 실행에서 선택·제외되는 이미지를 반드시 확인한다.

8. 로그와 알림

```
REMOTE_BOOT_ENABLE_HEALTH_LOGGING=true
REMOTE_BOOT_HEALTH_LOG_DIR="./logs/health"
REMOTE_BOOT_ALERT_STUB_LOG_FILE="./logs/alerts/remote_boot_alert_stub.log"
REMOTE_BOOT_ALERT_STATE_DIR="./state/alerts"
REMOTE_BOOT_LOG_FILE="/var/log/remote-boot.log"
REMOTE_BOOT_LOG_ROTATE_COUNT=14
```

변수	의미
REMOTE_BOOT_HEALTH_LOG_DIR	서버 상태 확인과 컨테이너 기동 후 점검의 실행별 로그 위치
REMOTE_BOOT_ALERT_STUB_LOG_FILE	Slack을 사용할 수 없을 때 실패를 남길 대체 로그
REMOTE_BOOT_ALERT_STATE_DIR	같은 실패의 반복 전송을 억제하는 상태 파일 위치

변수	의미
REMOTE_BOOT_LOG_FILE	systemd service의 전체 stdout-stderr 로그
REMOTE_BOOT_LOG_ROTATE_COUNT	daily logrotate 보존 개수

Slack 알림을 사용하려면 다음 값을 설정한다.

```
REMOTE_BOOT_SLACK_ENABLED=true
REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM="<Slack webhook URL>"
REMOTE_BOOT_SLACK_MESSAGE_PREFIX="[remote_boot]"
```

LAB/FARM 알림은 현재 하나의 FARM webhook을 사용하며, 서버 ID는 메시지에 반영된다. 최종 Slack 전송은 source에 고정된 내부 알림 API를 통한다. webhook은 secret이므로 terminal 출력, 예제 파일이나 Git commit에 남기지 않는다.

설정 후 실제 message를 확인한다.

```
./test/test_slack_notification.sh --server-id FARM1
```

9. 독립 시험용 컨테이너 설정

REMOTE_BOOT_TEST_* 변수는 create_test_container.sh와 delete_test_container.sh를 수동으로 사용할 때만 적용된다. WOL → 서버 준비 상태 확인 → 기존 컨테이너 시작으로 이어지는 기본 부팅 흐름에서는 읽지 않는다.

독립 시험 도구는 Docker image, UID/GID, memory, runtime과 mount를 실제로 사용할 수 있으므로 운영 부팅 검증과 구분해 별도 시험 계정과 resource로 실행한다.

10. 첫 설정 검증 순서

1. 로컬 설정 파일의 권한이 0600 인지 확인한다.
2. --list-targets 로 FARM/LAB 목록을 확인한다.
3. 각 대상 서버의 Ansible ping을 확인한다.
4. WOL 모의 실행에서 MAC과 broadcast IP를 확인한다.
5. 서버 상태 모의 실행에서 mount source와 경로를 확인한다.
6. 컨테이너 모의 실행에서 대상 이미지와 기동 후 점검 계획을 확인한다.
7. Slack을 사용하는 경우 시험 메시지를 전송한다.
8. 단일 서버에서 실제 WOL, 서버 상태 확인과 컨테이너 기동 후 점검을 순서대로 실행한다.
9. 마지막으로 systemd 설치 스크립트를 실행한다.

명령과 실제 상태 변경 범위는 운영 문서를 따른다.